



MCDI500

Evaluación Sumativa 3 — Fase 3

Núcleo algorítmico, eficiencia e implementación orientada a objetos

Título del proyecto:

Factores socioeconómicos y de preparación previa asociados al rendimiento académico

Integrantes: *Juan de Dios Díaz Ríos*

Francisco Fariña Molina

Constanza Moreno Giacometto

Yenne Sepúlveda Jerez

Curso: *MCDI500 Programación para la Ciencia de Datos*

Docente: *Omar Salinas Silva*

Fecha: *13/06/2026*

Repositorio: *<https://github.com/ffarina11/proyecto-grupo7-mcdi500>*

ÍNDICE

I. Diseño de soluciones algorítmicas eficientes	3
a. Codificación funcional y arquitectura básica del script	3
b. Preprocesamiento y transformación del dataset	5
c. Validación técnica y verificación del código	8
d. Eficiencia y optimización.....	12
e. Diseño estructurado del código	14
II. Implementación de código modular y robusto	15
a. Programación orientada a objetos	15
El carácter de contrato de este método se valida explícitamente intentando invocarlo directamente sobre la clase base, sin pasar por ninguna subclase, lo que produce una excepción controlada:.....	17
b. Documentación de arquitectura y funcionalidad del código	18
III. Repositorio GitHub (F3)	23
IV. Notebooks ejecutables (F3)	27
V. Bibliografía (APA 7)	33

I. Diseño de soluciones algorítmicas eficientes

a. Codificación funcional y arquitectura básica del script

La solución desarrollada para la Fase 3 implementa una arquitectura modular basada en principios de programación orientada a objetos y reutilización de componentes previamente desarrollados en la Fase 2 (Salinas, 2026a). El notebook **F3_S02_Nucleo_Algoritmico_POO.ipynb** actúa como punto de orquestación del proceso, integrando los módulos `functions.py`, `clases.py` y `recursivas.py`, cada uno especializado en una responsabilidad específica. Esta separación evita la duplicación de código y favorece la mantenibilidad, escalabilidad y reutilización de la solución.

```
# =====
# ENTORNO E IMPORTACIÓN DE MÓDULOS DEL PROYECTO
# =====
import sys, os
sys.path.append(os.path.abspath("../src"))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import timeit

warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (10, 4)
plt.rcParams['figure.dpi'] = 100
sns.set_theme(style='whitegrid', palette='muted')

# Funciones de La Fase 2 (pipeline funcional ya validado)
from functions import (
    cargar_dataset, mostrar_primeras_filas, resumen_dataset,
    validar_rangos, detectar_outliers_iqr, analizar_correlaciones,
    analizar_g3_cero, limpiar_dataset, winsorizacion,
    codificar_binarias, codificar_ohe, crear_variables_derivadas,
    validar_dataset_final, exportar_dataset, integrar_datasets,
)

# Núcleo algorítmico de La Fase 3: clases (POO) y funciones recursivas
from clases import PreprocesadorAsignatura, PreprocesadorMatematicas, PreprocesadorPortugues
from recursivas import listar_estructura, predecir_arbol, predecir_arbol_vectorizado, ARBOL_DECISION_APROBACION

print("✓ Entorno listo. Módulos de Fase 2 (functions) y Fase 3 (clases, recursivas) importados.")
```

✓ Entorno listo. Módulos de Fase 2 (functions) y Fase 3 (clases, recursivas) importados.

Las funciones y métodos implementados presentan parámetros explícitos y salidas claramente definidas, permitiendo comprender de manera directa las entradas requeridas y los resultados generados. El diseño sigue el principio de responsabilidad única, donde cada componente realiza una tarea específica dentro del pipeline. En particular, la clase base `PreprocesadorAsignatura` encapsula las operaciones de carga, exploración, limpieza, transformación, validación y exportación de datos, reutilizando las funciones desarrolladas durante la fase anterior sin modificar su implementación original.

```

# =====
# CLASE BASE
# =====
class PreprocesadorAsignatura:
    """Clase base que encapsula el pipeline completo de la Fase 2
    (carga -> exploración -> limpieza -> transformación -> validación
    -> exportación) para UNA asignatura del Student Performance Dataset.

    Atributos de instancia
    -----
    ruta : str
        Ruta al CSV crudo (data/raw/...).
    asignatura : str
        Identificador corto de la asignatura ('mat' o 'por').
    df_raw : pd.DataFrame | None
        Copia inmutable de los datos originales (tal cual se cargan).
    df : pd.DataFrame | None
        Copia de trabajo: se actualiza en limpiar().
    df_enc : pd.DataFrame | None
        Versión codificada/transformada (resultado de transformar()).

    Atributo de clase
    -----
    NOMBRE_ASIGNATURA : str
        Nombre legible de la asignatura. Se sobrescribe en cada
        clase hija (PreprocesadorMatematicas, PreprocesadorPortugues).
    """

    NOMBRE_ASIGNATURA = "Genérica"

    def __init__(self, ruta: str, asignatura: str):
        # __init__ es el constructor: deja listos los atributos del objeto.
        self.ruta = ruta
        self.asignatura = asignatura
        self.df_raw = None
        self.df = None
        self.df_enc = None

```

Imagen 1. Extracto de archivo *clases.py*

La Imagen 2 presenta un extracto del archivo *clases.py*, donde es posible observar la definición de métodos con parámetros claramente especificados, anotaciones de tipo (*type hints*), documentación interna mediante *docstrings* y una estructura de código legible y organizada, elementos que contribuyen a la mantenibilidad y comprensión de la solución.

Desde el punto de vista de la arquitectura del código, la solución implementa una separación explícita de responsabilidades entre módulos, funciones y clases. Cada función recibe parámetros claramente definidos y retorna resultados específicos, evitando dependencias innecesarias entre componentes. Esta estructura facilita la validación del flujo de datos durante la ejecución, ya que cada etapa puede ser evaluada de forma independiente mediante pruebas y verificaciones intermedias. Asimismo, la organización modular permite identificar con facilidad el origen y destino de la información procesada, favoreciendo la trazabilidad y depuración del sistema.

La implementación también considera buenas prácticas de desarrollo orientadas a la mantenibilidad del software. Se utilizan nombres descriptivos para variables, funciones, métodos y clases, junto con una estructura coherente de archivos que facilita la navegación y comprensión del proyecto. Los comentarios y docstrings incorporados documentan tanto el propósito de los componentes como los parámetros de entrada y valores de retorno, permitiendo que otros desarrolladores comprendan y reutilicen el código sin necesidad de revisar detalladamente la lógica interna de cada procedimiento.

b. Preprocesamiento y transformación del dataset

El preprocesamiento tuvo como objetivo transformar los datasets originales de estudiantes en conjuntos de datos consistentes, íntegros y aptos para análisis estadístico y modelamiento predictivo. Para ello se implementó un pipeline secuencial de procesamiento que integra las etapas de exploración, limpieza, transformación, validación y exportación de datos (The pandas development team, 2024). Cada etapa genera un resultado verificable que sirve de entrada para la siguiente, garantizando la trazabilidad de todas las modificaciones realizadas sobre la información original.

```
# 1) Creamos Los objetos (esto ejecuta __init__ de cada clase)
prep_mat = PreprocesadorMatematicas("../data/raw/student-mat.csv")
prep_por = PreprocesadorPortugues("../data/raw/student-por.csv")

print(repr(prepare_mat))
print(repr(prepare_por))

<PreprocesadorMatematicas asignatura='mat' filas=0 columnas_encoded=0>
<PreprocesadorPortugues asignatura='por' filas=0 columnas_encoded=0>

# 2) Ejecutamos el pipeline completo para Matemáticas
ok_mat = prep_mat.ejecutar_pipeline(exportar_resultados=True)

=====
PIPELINE - Matemáticas (objeto: PreprocesadorMatematicas)
=====
✓ Matemáticas: 395 filas x 33 columnas cargadas desde '../data/raw/student-mat.csv'

Primeras filas:
  school sex  age address famsize Pstatus  Medu  Fedu    Mjob    Fjob  ... \
0    GP   F   18     U    GT3      A      4      4  at_home  teacher  ...
1    GP   F   17     U    GT3      T      1      1  at_home   other  ...
2    GP   F   15     U    LE3      T      1      1  at_home   other  ...
3    GP   F   15     U    GT3      T      4      2  health  services  ...
4    GP   F   16     U    GT3      T      3      3   other    other  ...

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4         3      4      1      1      3      6  5  6  6
1      5         3      3      1      1      3      4  5  5  6
2      4         3      2      2      3      3     10  7  8  10
3      3         2      2      1      1      5      2  15  14  15
4      4         3      2      1      2      5      4  6  10  10

[5 rows x 33 columns]
```

Imagen 2. Llamada clase hija

```
# 3) Ejecutamos el pipeline completo para Portugués
```

```
ok_por = prep_por.ejecutar_pipeline(exportar_resultados=True)
```

```
=====
PIPELINE – Portugués (objeto: PreprocesadorPortugues)
=====
✓ Portugués: 649 filas x 33 columnas cargadas desde '../data/raw/student-por.csv'

Primeras filas:
  school sex  age address famsize Pstatus  Medu  Fedu  Mjob  Fjob  ... \
0    GP   F   18      U    GT3      A    4    4  at_home  teacher  ...
1    GP   F   17      U    GT3      T    1    1  at_home   other  ...
2    GP   F   15      U   LE3      T    1    1  at_home   other  ...
3    GP   F   15      U    GT3      T    4    2  health  services  ...
4    GP   F   16      U    GT3      T    3    3   other    other  ...

  famrel freetime  goout  Dalc  Walc health absences  G1  G2  G3
0      4         3      4      1      1      3      4      0  11  11
1      5         3      3      1      1      3      2      9  11  11
2      4         3      2      2      3      3      6     12  13  12
3      3         2      2      1      1      5      0     14  14  14
4      4         3      2      1      2      5      0     11  13  13

[5 rows x 33 columns]
```

Imagen 3. Llamada clase hija (parte2)

A diferencia de la Fase 2, donde se diseñaron y validaron individualmente las funciones de procesamiento, en esta etapa el énfasis se encuentra en su ejecución automatizada y reutilización dentro de una estructura orientada a objetos. De esta manera, las transformaciones previamente desarrolladas se aplican de forma consistente tanto al dataset de Matemáticas como al de portugués, garantizando uniformidad en el tratamiento de los datos.

El flujo de datos es completamente verificable y sigue una secuencia lógica de procesamiento:

Método	Qué hace	Funciones de Fase 2 que reutiliza
Cargar ()	Lee el CSV crudo y crea <code>self.df_raw</code> y <code>self.df</code>	<code>cargar_dataset()</code>
Explorar ()	Diagnóstico inicial: resumen, rangos, outliers y registros con G3 = 0	<code>resumen_dataset()</code> , <code>estadisticas_descriptivas()</code> , <code>plot_distribucion_categoricas()</code> , <code>validar_rangos()</code> , <code>detectar_outliers_iqr()</code> , <code>analizar_g3_cero()</code>

Limpiar ()	Genera indicadores y aplica winsorización de ausencias	<code>limpiar_dataset(), winsorizacion()</code>
Transformar ()	Codificación binaria, One-Hot Encoding y variables derivadas	<code>codificar_binarias(), codificar_ohe(), crear_variables_derivadas()</code>
Validar ()	Verificaciones de integridad posteriores al procesamiento	<code>validar_dataset_final()</code>
Exportar ()	Exportación de datasets procesados	<code>exportar_dataset()</code>

Durante la exploración se verificó nuevamente la estructura de los datos, los tipos de variables, la presencia de posibles valores atípicos y la existencia de valores ausentes (NA). Tal como se documentó en la Fase 2, los datasets originales no presentan valores nulos, condición que se mantiene y es posteriormente comprobada mediante validaciones automáticas antes de exportar los resultados finales.

En la etapa de limpieza se reutilizaron las transformaciones desarrolladas en la Fase 2 para identificar estudiantes con posibles situaciones de deserción académica mediante la creación de una variable indicadora de deserción. Esta variable permite distinguir registros que presentan características asociadas al abandono o interrupción de los estudios, conservando la información original y agregando una nueva característica potencialmente relevante para el análisis posterior. Adicionalmente, se aplicó winsorización sobre la variable `absences` con el objetivo de reducir la influencia de valores extremos sin eliminar observaciones del conjunto de datos.

Posteriormente, la fase de transformación aplica las conversiones necesarias para adaptar los datos a formatos compatibles con técnicas de análisis y aprendizaje automático. Entre ellas se incluyen la codificación binaria de variables categóricas, la aplicación de One-Hot Encoding para variables nominales con múltiples categorías y la generación de variables derivadas orientadas a enriquecer la representación de la información académica y socioeducativa de los estudiantes.

Respecto a la normalización, no fue necesario aplicar procedimientos adicionales durante esta fase, debido a que las variables numéricas del dataset presentan rangos acotados y homogéneos. Además, la eventual necesidad de normalización depende del algoritmo específico utilizado durante el modelamiento, por lo que dicha decisión puede abordarse posteriormente según los requerimientos de cada técnica de aprendizaje.

Finalmente, la etapa de validación ejecuta verificaciones automáticas destinadas a garantizar la consistencia de los datos procesados. Estas comprobaciones incluyen la ausencia de valores nulos, la correcta codificación de variables binarias, la validez de los rangos de las calificaciones y la coherencia estructural del dataset resultante. Solo después de superar estas verificaciones los datasets son exportados para su utilización en las etapas posteriores del proyecto.

Validando: Matemáticas

- ✓ Filas conservadas: 395
- ✓ Sin nulos en columnas originales
- ✓ G1, G2, G3 intactos
- ✓ Flag 'aprobado' coherente (265 aprobados)
- ✓ Flag 'desercion' coherente (13 casos)
- ✓ Sin filas duplicadas
- ✓ G1, G2, G3 en rango [0, 20]
- VALIDACIÓN EXITOSA 
- ✓ student_mat_clean.csv 395f × 36c (45.6 KB)
 - Matemáticas limpio: flags + winsorización (sin OHE)
- ✓ student_mat_clean_encode.csv 395f × 54c (48.5 KB)
 - Matemáticas limpio + encoded: flags + winsorización + OHE + variables derivadas

Validando: Portugués

- ✓ Filas conservadas: 649
- ✓ Sin nulos en columnas originales
- ✓ G1, G2, G3 intactos
- ✓ Flag 'aprobado' coherente (549 aprobados)
- ✓ Flag 'desercion' coherente (7 casos)
- ✓ Sin filas duplicadas
- ✓ G1, G2, G3 en rango [0, 20]
- VALIDACIÓN EXITOSA 
- ✓ student_por_clean.csv 649f × 36c (72.8 KB)
 - Portugués limpio: flags + winsorización (sin OHE)
- ✓ student_por_clean_encode.csv 649f × 54c (77.7 KB)
 - Portugués limpio + encoded: flags + winsorización + OHE + variables derivadas

c. Validación técnica y verificación del código

La validación técnica del núcleo algorítmico se realizó mediante un conjunto de verificaciones orientadas a garantizar la correcta ejecución del pipeline, la consistencia de los datos procesados y la correcta implementación de los algoritmos desarrollados. Estas verificaciones consideran casos normales de funcionamiento, casos límite, manejo de excepciones, pruebas automatizadas y mecanismos de trazabilidad que permiten seguir el flujo completo de procesamiento.

Verificación de casos normales, límite y excepciones

La primera etapa de validación consistió en comprobar el comportamiento esperado del sistema bajo condiciones normales de ejecución. Para ello, se ejecutó el pipeline completo sobre ambos conjuntos de datos, verificando que cada una de las etapas definidas en la arquitectura (carga, exploración, limpieza, transformación, validación y exportación) se ejecutara de forma secuencial y sin errores. Los mensajes registrados durante la ejecución permiten verificar que el flujo se completó correctamente y que cada etapa entregó resultados válidos a la siguiente..

```
# 1) Creamos Los objetos (esto ejecuta __init__ de cada clase)
prep_mat = PreprocesadorMatematicas("../data/raw/student-mat.csv")
prep_por = PreprocesadorPortugues("../data/raw/student-por.csv")

print(repr(prepare_mat))
print(repr(prepare_por))

<PreprocesadorMatematicas asignatura='mat' filas=0 columnas_encoded=0>
<PreprocesadorPortugues asignatura='por' filas=0 columnas_encoded=0>
```

```
# 2) Ejecutamos el pipeline completo para Matemáticas
ok_mat = prep_mat.ejecutar_pipeline(exportar_resultados=True)
```

```
=====
PIPELINE - Matemáticas (objeto: PreprocesadorMatematicas)
=====
✓ Matemáticas: 395 filas x 33 columnas cargadas desde '../data/raw/student-mat.csv'
```

Primeras filas:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	\
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	
3	GP	F	15	U	GT3	T	4	2	health	services	...	
4	GP	F	16	U	GT3	T	3	3	other	other	...	

	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	4	3	4	1	1	3	6	5	6	6
1	5	3	3	1	1	3	4	5	5	6
2	4	3	2	2	3	3	10	7	8	10
3	3	2	2	1	1	5	2	15	14	15
4	4	3	2	1	2	5	4	6	10	10

[5 rows x 33 columns]

```
# 3) Ejecutamos el pipeline completo para Portugués
ok_por = prep_por.ejecutar_pipeline(exportar_resultados=True)
```

```
=====
PIPELINE - Portugués (objeto: PreprocesadorPortugues)
=====
✓ Portugués: 649 filas x 33 columnas cargadas desde '../data/raw/student-por.csv'
```

Primeras filas:

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	Mjob	Fjob	...	\
0	GP	F	18	U	GT3	A	4	4	at_home	teacher	...	
1	GP	F	17	U	GT3	T	1	1	at_home	other	...	
2	GP	F	15	U	LE3	T	1	1	at_home	other	...	
3	GP	F	15	U	GT3	T	4	2	health	services	...	
4	GP	F	16	U	GT3	T	3	3	other	other	...	

	famrel	freetime	goout	Dalc	Walc	health	absences	G1	G2	G3
0	4	3	4	1	1	3	4	0	11	11
1	5	3	3	1	1	3	2	9	11	11
2	4	3	2	2	3	3	6	12	13	12
3	3	2	2	1	1	5	0	14	14	14
4	4	3	2	1	2	5	0	11	13	13

[5 rows x 33 columns]

Además de los casos normales, durante el diseño y validación del pipeline se consideraron situaciones límite que podrían afectar la calidad de los datos y los resultados obtenidos. Entre ellas se encuentran registros con calificaciones extremas, valores atípicos asociados a las inasistencias y otras condiciones que requieren un tratamiento específico para evitar distorsiones en las etapas posteriores de análisis.

La gestión de estos casos se realiza mediante las reglas de limpieza y transformación implementadas en el proceso de preprocesamiento, las cuales permiten conservar la información relevante y, al mismo tiempo, garantizar la consistencia del conjunto de datos resultante. La verificación detallada de estas condiciones fue desarrollada y documentada durante la fase anterior del proyecto, por lo que en esta etapa se reutilizan los procedimientos ya validados e integrados dentro de la arquitectura orientada a objetos.

De esta forma, la validación de casos límite no depende de intervenciones manuales posteriores, sino que forma parte del flujo normal de ejecución del pipeline, asegurando un tratamiento consistente de los datos para ambas asignaturas procesadas.

Por otra parte, la robustez de la arquitectura orientada a objetos fue validada mediante pruebas de excepción. La clase base define métodos abstractos que deben ser implementados por las subclases especializadas. Cuando se intenta utilizar directamente uno de estos métodos sin sobrescribir, el sistema genera una excepción controlada de tipo `NotImplementedError`, asegurando el cumplimiento de los contratos de diseño establecidos (Salinas, 2026d).

```
# La clase base no implementa interpretar_resultados(): debe sobrescribirse en la subclase
base = PreprocesadorAsignatura(ruta="../../data/raw/student-mat.csv", asignatura='mat')
try:
    base.interpretar_resultados()
except NotImplementedError as e:
    print(f"NotImplementedError (esperado): {e}")

NotImplementedError (esperado): interpretar_resultados() debe implementarse en la subclase correspondiente (PreprocesadorMatematicas / PreprocesadorPortugues).
```

Comprobación de resultados intermedios

La verificación no se limitó únicamente a los resultados finales. Durante la ejecución se inspeccionaron estados intermedios del procesamiento con el objetivo de confirmar que las transformaciones se aplicaran correctamente sobre los datos. Esta revisión permitió verificar la creación de variables derivadas, la correcta codificación de atributos categóricos y la preservación de la estructura esperada del conjunto de datos.

```
# La tabla final de cada asignatura vive dentro del objeto correspondiente
print("Vista previa - Matemáticas (prep_mat.df_enc):")
mostrar_primeras_filas(prepare_mat.df_enc, 5)
```

Vista previa - Matemáticas (prep_mat.df_enc):

	school	sex	age	address	famsize	Pstatus	Medu	Fedu	traveltime	studytime	...	Fjob_teacher	reason_home	reason_other	reason_reputation	guardian
0	GP	F	18	U	GT3	A	4	4	2	2	...	1	0	0	0	
1	GP	F	17	U	GT3	T	1	1	1	2	...	0	0	0	0	
2	GP	F	15	U	LE3	T	1	1	1	2	...	0	0	1	0	
3	GP	F	15	U	GT3	T	4	2	1	3	...	0	1	0	0	
4	GP	F	16	U	GT3	T	3	3	1	2	...	0	1	0	0	

5 rows × 54 columns

Pruebas manuales y automatizadas

La validación también incorporó pruebas automáticas basadas en sentencias assert, las cuales verifican condiciones críticas para la integridad del sistema. Estas comprobaciones incluyen la ausencia de valores nulos, la consistencia de las variables binarias generadas, la validez de los rangos de las calificaciones y la conservación de las dimensiones esperadas de los conjuntos de datos procesados.

Adicionalmente, se realizaron pruebas de correctitud sobre los algoritmos implementados en la fase. En particular, la salida del algoritmo recursivo Merge Sort fue comparada con los resultados obtenidos mediante la función nativa `sorted()` de Python, verificando que ambos métodos producen exactamente el mismo ordenamiento. De igual forma, las versiones iterativa y vectorizada de los procedimientos analizados fueron contrastadas para asegurar equivalencia funcional antes de evaluar su desempeño.

```
# --- Aplicado a Matemáticas ---  
g3_mat = prep_mat.df[["G3"]].dropna().tolist()  
print("--- Matemáticas - G3 ---")  
print("Antes :", g3_mat[:39])  
print("Después:", merge_sort(g3_mat)[:39])  
  
# --- Aplicado a Portugués ---  
g3_por = prep_por.df[["G3"]].dropna().tolist()  
print("\n--- Portugués - G3 ---")  
print("Antes :", g3_por[:20])  
print("Después:", merge_sort(g3_por)[:20])  
  
# --- Verificación ---  
assert merge_sort(g3_mat) == sorted(g3_mat), "¡Error en Merge Sort Mat!"  
assert merge_sort(g3_por) == sorted(g3_por), "¡Error en Merge Sort Por!"  
print("\n✓ Verificación: merge_sort produce el mismo resultado que sorted().")
```

```
--- Matemáticas - G3 ---  
Antes : [6, 6, 10, 15, 10, 15, 11, 6, 19, 15, 9, 12, 14, 11, 16, 14, 14, 10, 5, 10, 15, 15, 16, 12, 8, 8, 11, 15, 11, 11, 12, 17, 16, 12, 15, 6,  
18, 15, 11]  
Después: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
--- Portugués - G3 ---  
Antes : [11, 11, 12, 14, 13, 13, 13, 13, 17, 13, 14, 13, 12, 13, 15, 17, 14, 14, 7, 12]  
Después: [0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

```
✓ Verificación: merge sort produce el mismo resultado que sorted().
```

Trazabilidad del proceso

Finalmente, la solución incorpora mecanismos que permiten reconstruir y verificar el proceso completo de ejecución. Los registros generados durante el pipeline, los estados de los objetos creados y los archivos exportados constituyen evidencia verificable de cada una de las etapas realizadas. Esta trazabilidad facilita la auditoría de resultados, la reproducibilidad del análisis y la identificación de posibles incidencias durante futuras ejecuciones.

```

=====
RESUMEN – NÚCLEO ALGORÍTMICO POO (FASE 3)
=====

OBJETOS CREADOS
  prep_mat = <PreprocesadorMatematicas asignatura='mat' filas=395 columnas_encoded=54>
  prep_por = <PreprocesadorPortugues asignatura='por' filas=649 columnas_encoded=54>

PIPELINE ENCAPSULADO (heredado de PreprocesadorAsignatura)
  cargar() -> explorar() -> limpiar() -> transformar() -> validar() -> exportar()

HERENCIA Y POLIMORFISMO
  PreprocesadorMatematicas.interpretar_resultados() -> análisis específico MAT
  PreprocesadorPortugues.interpretar_resultados()   -> análisis específico POR

RECURSIVIDAD
  merge_sort() -> ordenar las listas dividiendolas en 2
  combinar()   -> une ambas listas ordenadas

EFICIENCIA
  determinar_aprobados: vectorizado ~204x más rápido que iterrows

DATOS EXPORTADOS -> data/processed/F3/
  student_mat_clean.csv          395 x 36
  student_mat_clean_encode.csv   395 x 54
  student_por_clean.csv          649 x 36
  student_por_clean_encode.csv   649 x 54
  student_union_clean_encode.csv 1044 x 54
=====

```

Como evidencia adicional de la correcta finalización del proceso, los archivos exportados en la carpeta de resultados permiten comprobar que los datasets procesados fueron generados exitosamente y se encuentran disponibles para las etapas posteriores del proyecto.

d. Eficiencia y optimización

El análisis de eficiencia se desarrolló comparando formalmente dos pares de implementaciones equivalentes en su resultado, pero distintas en su forma de recorrer los datos, utilizando el módulo `timeit` de la biblioteca estándar de Python para obtener mediciones reproducibles (Python Software Foundation, 2026).

La primera comparación evalúa la determinación de estudiantes aprobados ($G3 \geq 10$) sobre el dataset de Portugués ($n = 649$), contrastando una versión iterativa basada en `iterrows()` con una versión vectorizada de pandas. Se verificó la equivalencia funcional de ambas implementaciones:

Comparamos dos formas de calcular si un estudiante aprobó ($63 \geq 10$):

- **Versión A — bucle:** recorre fila por fila con `iterrows()` → $O(n)$
- **Versión B — vectorizada:** usa pandas sobre todo el array de una vez → también $O(n)$ pero con constante mucho menor porque opera en C/NumPy en vez de Python puro.

Medimos con `timeit` para respaldar la afirmación con datos reales.

```
def aprobado_bucle(df: pd.DataFrame) -> list:
    """
    Determina si cada estudiante aprobó ( $63 \geq 10$ ) usando un bucle fila a fila.
    Complejidad:  $O(n)$  pero lento por el overhead de Python en cada iteración.
    """
    resultado = []
    for _, fila in df.iterrows():
        # iterrows es lento: crea un objeto por fila
        resultado.append(1 if fila["G3"] >= 10 else 0)
    return resultado

def aprobado_vectorizado(df: pd.DataFrame) -> pd.Series:
    """
    Determina si cada estudiante aprobó ( $63 \geq 10$ ) con operación vectorizada.
    Complejidad:  $O(n)$  pero rápido porque opera en C/NumPy de una sola vez.
    """
    return (df["G3"] >= 10).astype(int)

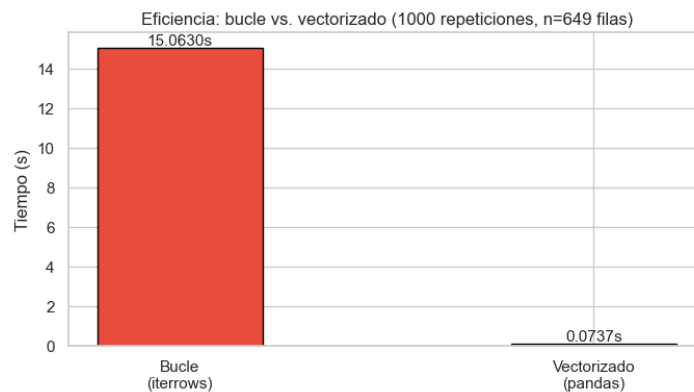
# Verificación de equivalencia
res_bucle = aprobado_bucle(prepare_data)
res_vec = aprobado_vectorizado(prepare_data).tolist()
assert res_bucle == res_vec, "¡Los resultados difieren!"
print("✓ Ambas versiones producen el mismo resultado")

# Medición de tiempos
N = 1000 # repeticiones para timeit
t_bucle = timeit.timeit(lambda: aprobado_bucle(prepare_data), number=N)
t_vec = timeit.timeit(lambda: aprobado_vectorizado(prepare_data), number=N)

print(f"Medición con timeit ((N) repeticiones | n={len(prepare_data)} filas):")
print(f"  Bucle      : {t_bucle:.4f} s")
print(f"  Vectorizado : {t_vec:.4f} s")
print(f"  El vectorizado fue ~{(t_bucle/t_vec:.0f)}x más rápido")
print("\nInterpretación: ambas son  $O(n)$ , pero la versión vectorizada opera")
print("en C/NumPy sin overhead de Python por fila, por eso es mucho más rápida.")
print("En ciencia de datos siempre se prefieren operaciones vectorizadas sobre bucles.")
```

✓ Ambas versiones producen el mismo resultado

La medición se realizó con 1.000 repeticiones sobre las 649 filas del dataset, registrándose los tiempos mediante `timeit.timeit()`. El resultado se visualiza en el siguiente gráfico, generado directamente a partir de las mediciones obtenidas (Hunter, 2007):



El resultado evidencia una diferencia de aproximadamente 204 veces a favor de la versión vectorizada. Ambas implementaciones comparten la misma complejidad temporal asintótica, $O(n)$, ya que cada fila del dataset se procesa exactamente una vez en los dos casos; también comparten la misma complejidad espacial, $O(n)$, dado que ambas almacenan un resultado de tamaño equivalente al número de filas (una lista en el caso del bucle, una Series de pandas en el caso vectorizado). La diferencia observada no se explica entonces por el orden de crecimiento, sino por la constante que multiplica a n : `iterrows()` construye un objeto Series por cada fila e introduce overhead de interpretación de Python en cada iteración, mientras que la versión vectorizada delega el recorrido completo a código compilado en C, operando directamente sobre los arreglos de memoria contigua de NumPy (Harris et al., 2020; NumPy Developers, 2024).

Esta comparación justifica la elección de operaciones vectorizadas y de una arquitectura basada en objetos como las alternativas más eficientes disponibles dentro del proyecto, sin sacrificar la legibilidad ni la mantenibilidad del código.

e. Diseño estructurado del código

El diseño estructurado del código se evidencia en la separación clara entre los componentes del pipeline (encapsulados en la clase PreprocesadorAsignatura y sus métodos) y el componente recursivo, implementado de forma independiente mediante funciones que no dependen del estado de ningún objeto.

Antes de implementar el algoritmo concreto, el notebook presenta el esqueleto general de cualquier algoritmo recursivo tipo divide y vencerás, separando explícitamente el caso base del caso recursivo (Cormen et al., 2022):

```
def algoritmo_recursivo(datos):
    # 1) CASO BASE: cuando paran (datos muy pequeños) -> return
    if len(datos) <= 1:
        return datos
    # 2) CASO RECURSIVO: divide el problema y vuelve a llamarse
    mitad = len(datos) // 2
    izq = algoritmo_recursivo(datos[:mitad])
    der = algoritmo_recursivo(datos[mitad:])
    return izq + der # placeholder: aquí iría la lógica de combinación
```

Sobre esa misma estructura se construye la implementación concreta, Merge Sort, aplicada a la variable G3 de ambos datasets:

```
def merge_sort(lista):
    """
    Ordena una lista usando Merge Sort recursivo (divide y vencerás,  $O(n \log n)$ ).
    CASO BASE : lista de 0 o 1 elemento ya ordenada, se retorna tal cual.
    CASO RECURSIVO: divide la lista a la mitad, ordena cada parte
    recursivamente y combina los resultados.
    """
    if len(lista) <= 1: # CASO BASE: lista mínima, nada que ordenar
        return lista

    medio = len(lista) // 2
    izq = merge_sort(lista[:medio]) # divide y ordena mitad izquierda
    der = merge_sort(lista[medio:]) # divide y ordena mitad derecha
    return combinar(izq, der) # y vence (combina)

def combinar(a, b):
    """Fusiona dos listas ya ordenadas en una sola lista ordenada."""
    res, i, j = [], 0, 0
    while i < len(a) and j < len(b):
        if a[i] <= b[j]:
            res.append(a[i]); i += 1
        else:
            res.append(b[j]); j += 1
    res.extend(a[i:]); res.extend(b[j:])
    return res
```

El flujo de datos entre ambas funciones está documentado explícitamente mediante docstrings que especifican el caso base, el caso recursivo y el rol de la función auxiliar combinar(). La función merge_sort() presenta alta cohesión: su única responsabilidad es dividir la lista y delegar el ordenamiento de cada mitad a llamadas recursivas de sí misma. combinar(), a su vez, tiene una responsabilidad igualmente acotada: fusionar dos listas ya ordenadas en una sola. El bajo acoplamiento entre ambas se refleja en que combinar() no depende de merge_sort() más allá de recibir dos listas como parámetros, y en que ninguna de las dos funciones depende de los objetos PreprocesadorAsignatura ni de pandas (Richards & Ford, 2025): ambas operan sobre listas genéricas de Python, lo que les permite ser reutilizadas sobre cualquier estructura de datos ordenable, no solo sobre la variable G3.

Esta independencia respecto del resto de la arquitectura también aporta a la escalabilidad y extensibilidad del diseño: el mismo esqueleto recursivo presentado en la Sección 5.1 podría reutilizarse para implementar otros algoritmos de tipo divide y vencerás sobre el proyecto, sin necesidad de modificar la clase PreprocesadorAsignatura ni los pipelines ya construidos.

II. Implementación de código modular y robusto

a. Programación orientada a objetos

La arquitectura orientada a objetos del proyecto se organiza alrededor de una clase base, PreprocesadorAsignatura, que encapsula el pipeline completo de la Fase 2 en seis métodos de responsabilidad única (cargar, explorar, limpiar, transformar, validar y exportar), más un método de alto nivel, ejecutar_pipeline(), que orquesta a los seis en el orden correcto. En la siguiente imagen se captura, una muestra del archivo clases.py (que se encuentra en la carpeta SRC) que muestra la encapsulación del pipeline completo de la fase 2.

```
class PreprocesadorAsignatura:
    """Clase base que encapsula el pipeline completo de la Fase 2
    (carga -> exploración -> limpieza -> transformación -> validación
    -> exportación) para UNA asignatura del Student Performance Dataset.
```

```

def __init__(self, ruta: str, asignatura: str):
    # __init__ es el constructor: deja listos los atributos del objeto.
    self.ruta = ruta
    self.asignatura = asignatura
    self.df_raw = None
    self.df = None
    self.df_enc = None

# -----
# 1. CARGA
# -----
def cargar(self) -> pd.DataFrame:
    """Carga el CSV crudo y guarda una copia inmutable (df_raw) y
    una copia de trabajo (df).

    Nota: `cargar_dataset()` (Fase 2) solo lee el CSV y retorna el DataFrame,
    """
    self.df_raw = cargar_dataset(self.ruta)
    self.df = self.df_raw.copy()
    print(f" {self.NOMBRE_ASIGNATURA}: {self.df.shape[0]} filas "
          f" {self.df.shape[1]} columnas cargadas desde '{self.ruta}'")
    return self.df

# -----
# 2. EXPLORACIÓN
# -----
def explorar(self, mostrar_correlaciones: bool = False) -> dict:
    """Ejecuta el diagnóstico completo sobre self.df: resumen general,
    validación de rangos, outliers (IQR) y análisis de G3=0.

    Nota: `resumen_dataset()` (Fase 2) solo imprime y retorna None,
    por lo que el diccionario-resumen se construye aquí directamente
    a partir de self.df (mismas métricas que imprime resumen_dataset).
    """

```

Cada método recibe parámetros explícitos, retorna un resultado específico y está documentado mediante docstrings que describen su propósito. La clase presenta alta cohesión, ya que todos sus métodos giran en torno a una única responsabilidad —procesar los datos de una asignatura— y bajo acoplamiento respecto de las clases hijas, dado que la clase base no necesita conocer los detalles particulares de Matemáticas o Portugués para funcionar.

El encapsulamiento se evidencia en que el estado del objeto (df_raw, df, df_enc) solo se modifica a través de los métodos de la propia clase, y en que el constructor (__init__) deja ese estado correctamente inicializado desde el momento en que se crea el objeto (Salinas, 2026b):

Sobre esta base se construyen dos clases hijas mediante herencia, una por asignatura:

```

class PreprocesadorMatematicas(PreprocesadorAsignatura):
    NOMBRE_ASIGNATURA = "Matemáticas"

    def __init__(self, ruta="../data/raw/student-mat.csv"):
        super().__init__(ruta=ruta, asignatura='mat')

    def interpretar_resultados(self):
        ... # texto específico de Matemáticas

class PreprocesadorPortugues(PreprocesadorAsignatura):
    NOMBRE_ASIGNATURA = "Portugués"

    def __init__(self, ruta="../data/raw/student-por.csv"):
        super().__init__(ruta=ruta, asignatura='por')

    def interpretar_resultados(self):
        ... # texto específico de Portugués

```

Ambas clases reutilizan el constructor de la clase padre mediante super().__init__(), fijando únicamente su propia ruta de datos y el identificador de asignatura correspondiente. El atributo

de clase NOMBRE_ASIGNATURA se sobrescribe en cada subclase, de modo que todos los mensajes del pipeline heredado (resúmenes, validaciones, advertencias) muestren el nombre correcto sin duplicar código.

El polimorfismo se manifiesta en el método `interpretar_resultados()`, declarado en la clase base como un contrato que cada subclase debe implementar de forma distinta (Salinas, 2026c). Al invocarlo sobre cada objeto, el mismo nombre de método produce un comportamiento diferente según la clase concreta que lo ejecuta:

```
4.2 Probar el polimorfismo

Llamamos interpretar_resultados() sobre prep_mat y prep_por. El método se ve igual desde afuera, pero cada objeto ejecuta su propia versión.

# Polimorfismo: mismo metodo, comportamiento distinto segun la clase concreta
prep_mat.interpretar_resultados()
prep_por.interpretar_resultados()
```

```
=====
INTERPRETACIÓN - Matemáticas
=====

Las variables con mayor relación positiva con G3 corresponden a las
notas de periodos anteriores: G2 (0.90) y G1 (0.80), lo que indica
que el desempeño previo es el principal predictor del rendimiento final.

En menor medida, el nivel educativo de la madre (Medu=0.22) y del
padre (Fedu=0.15) muestran correlaciones positivas débiles.

La variable con mayor asociación negativa es 'failures' (-0.36):
los estudiantes con más asignaturas reprobadas previamente tienden
a obtener notas finales más bajas. Las variables de hábitos (goout,
Dalc, Walc) presentan asociaciones débiles.

=====
INTERPRETACIÓN - Portugués
=====

Igual que en Matemáticas, las notas previas dominan la correlación
con G3: G2 (0.92) y G1 (0.83) son los predictores más fuertes.

...
(Walc=-0.18) se asocia negativamente con el rendimiento, más
marcado que en Matemáticas. 'failures' (-0.39) sigue siendo la
variable con la correlación negativa más fuerte.

Output is truncated. View as a scrollable element or open in a text editor. Adjust cell output settings...
```

El carácter de contrato de este método se valida explícitamente intentando invocarlo directamente sobre la clase base, sin pasar por ninguna subclase, lo que produce una excepción controlada:

```
# La clase base no implementa interpretar_resultados(): debe sobrescribirse en la subclase
base = PreprocesadorAsignatura(ruta='../data/raw/student-mat.csv', asignatura='mat')
try:
    base.interpretar_resultados()
except NotImplementedError as e:
    print(f"NotImplementedError (esperado): {e}")
```

```
NotImplementedError (esperado): interpretar_resultados() debe implementarse en la subclase correspondiente (PreprocesadorMatematicas / PreprocesadorPortugues).
```

Este diseño resulta sostenible y extensible: incorporar una tercera asignatura al proyecto solo requeriría crear una nueva clase hija que extienda `PreprocesadorAsignatura` y que implemente su propia versión de `interpretar_resultados()`, sin necesidad de modificar el pipeline ya encapsulado en la clase base.

b. Documentación de arquitectura y funcionalidad del código

La solución desarrollada adopta una arquitectura por capas, separando la interacción con el usuario (notebook), la lógica orientada a objetos (clases.py), las funciones reutilizables (functions.py) y la fuente de datos (Student Performance Dataset). Adicionalmente, la clase PreprocesadorAsignatura implementa el patrón de diseño **Template Method**, ya que define la secuencia general del pipeline de procesamiento, delegando en las clases hijas la especialización del método polimórfico interpretar_resultados() (Cooper, 2021). Esta estructura favorece la modularidad, la reutilización y la mantenibilidad del código.

En cuanto a patrones de diseño, en la siguiente tabla se indican los utilizados:

Patrón / diseño	Dónde aparece	Ejemplo en tu código
Arquitectura por capas	notebook → clases.py → functions.py → dataset	clases.py reutiliza funciones validadas de functions.py, actuando como capa POO sobre el pipeline funcional.
Template Method	PreprocesadorAsignatura.ejecutar_pipeline()	Define una secuencia fija: cargar → explorar → limpiar → transformar → validar → exportar.
Herencia	PreprocesadorMatematicas y PreprocesadorPortugues	Ambas clases heredan de PreprocesadorAsignatura.
Polimorfismo	interpretar_resultados()	El mismo método existe en la clase base y se redefine con una interpretación distinta para Matemáticas y Portugués.
Facade / Fachada	ejecutar_pipeline()	Permite ejecutar todo el proceso con un solo método, ocultando la complejidad interna del pipeline.
Encapsulamiento	PreprocesadorAsignatura	El objeto mantiene su estado interno en df_raw, df y df_enc.

Representación de objeto	<code>__repr__()</code>	Entrega una descripción legible del estado del objeto: dataset cargado o transformado.
Algoritmo divide y vencerás	<code>merge_sort()</code>	Implementa ordenamiento recursivo dividiendo la lista y combinando resultados.

La arquitectura del notebook que refleja una secuencia real de llamadas entre notebook, clases y funciones, se visualiza de la siguiente manera:

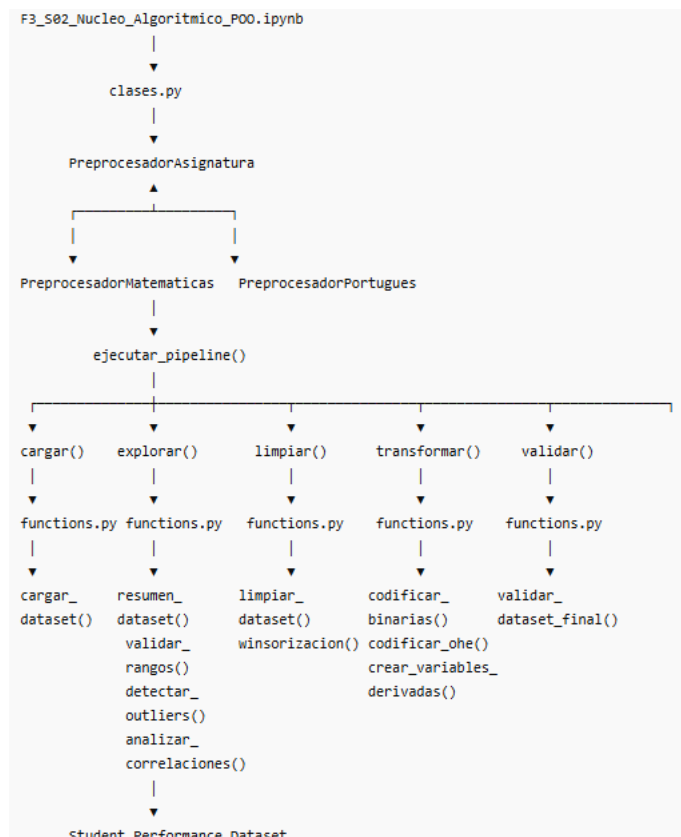


Imagen 4. Flujo de interacción entre notebook, clases, funciones y datos

Nota: la clase hija `PreprocesadorPortugues` se comporta igual que `PreprocesadorMatematicas`, sólo por dejar imagen limpia y clara no fue agregada.

Se decidió separar el código en notebooks, clases y funciones para evitar duplicación, mejorar la trazabilidad, reutilizar el pipeline validado en F2 y facilitar futuras extensiones del proyecto.

A continuación, se detallan fragmentos de código utilizados:

Ejemplo 1. Fragmento de código evidencia Clase, objeto, Constructor (`__init__`), Encapsulamiento, Herencia, Reutilización de código mediante `super()`

Clase	Descripción
<pre>class PreprocesadorAsignatura: def __init__(self, ruta: str, asignatura: str): # __init__ es el constructor: deja listos los atributos del objeto. self.ruta = ruta self.asignatura = asignatura self.df_raw = None self.df = None self.df_enc = None</pre>	El método especial <code>__init__()</code> actúa como constructor de la clase y se ejecuta automáticamente cuando se crea un objeto. Su función es inicializar el estado interno del objeto, almacenando la ruta del dataset y la asignatura correspondiente, además de preparar los atributos <code>df_raw</code>, <code>df</code> y <code>df_enc</code>, que contendrán respectivamente el conjunto de datos original, el conjunto de trabajo y la versión transformada del dataset. Ejemplo de uso: mat = PreprocesadorMatematicas() Al ejecutarse la instrucción, se llama al constructor de la clase hija PreprocesadorMatematicas
<pre>PreprocesadorMatematicas(PreprocesadorAsignatura): def __init__(self, ruta: str = "../data/raw/student-mat.csv"): super().__init__(ruta=ruta, asignatura='mat')</pre>	La llamada <code>super().__init__(ruta=ruta, asignatura='mat')</code> permite reutilizar el constructor de la clase padre, evitando duplicación de código y aprovechando el mecanismo de herencia de la Programación Orientada a Objetos.

Ejemplo 2. Fragmento de código método ejecutar_pipeline()

Este método de alto nivel encapsula la secuencia completa de procesamiento de una asignatura. Desde el notebook basta con instanciar un objeto y ejecutar una única llamada (`mat.ejecutar_pipeline()`), mientras que internamente el objeto coordina las etapas de carga, exploración, limpieza, transformación, validación y exportación. Esta implementación corresponde al patrón de diseño Template Method y permite ocultar la complejidad del flujo de trabajo, favoreciendo la reutilización y mantenibilidad del código.

Fragmento	Descripción
<pre>def ejecutar_pipeline(self, exportar_resultados=True, base_path="../data/processed/F3/"): pasos = [("cargar", self.cargar), ("explorar", self.explorar), ("limpiar", self.limpiar), ("transformar", self.transformar), ("validar", self.validar),] for nombre_paso, metodo in pasos: resultado = metodo() if exportar_resultados: self.exportar(base_path) return ok</pre>	La llamada <code>ok_mat = mat.ejecutar_pipeline()</code> desencadena la siguiente secuencia definida en la clase <code>PreprocesadorAsignatura</code>: <pre>graph TD A[ejecutar_pipeline()] --> B[cargar()] B --> C[explorar()] C --> D[limpiar()] D --> E[transformar()] E --> F[validar()] F --> G[exportar()]</pre>

Ejemplo 3. Fragmento de código método `__repr__()`

El método especial `__repr__()` fue implementado para proporcionar una representación legible del estado interno del objeto. En particular, permite conocer la asignatura asociada y el estado de los atributos `df` y `df_enc`, indicando si los conjuntos de datos han sido cargados o transformados. Esta funcionalidad facilita la depuración y seguimiento del pipeline durante su ejecución, proporcionando una descripción significativa del objeto en lugar de la representación predeterminada de Python

Fragmento	Descripción
<pre>def __repr__(self) -> str: if self.df is None: estado_df = "sin cargar" else: estado_df = f"{self.df.shape[0]} filas × {self.df.shape[1]} cols" if self.df_enc is None: estado_enc = "sin transformar" else: estado_enc = f"{self.df_enc.shape[0]} filas × {self.df_enc.shape[1]} cols" return (f"<{self.__class__.__name__} asignatura='{self.asignatura}' " f" df={estado_df} df_enc={estado_enc}>")</pre>	Después de crear el objeto, la utilización en el notebook, <code>mat = PreprocesadorMatematicas()</code>, ocurre la llamada mediante <code>print(mat)</code> Y ocurre la salida: <pre><PreprocesadorMatematicas asignatura='mat' df=395 filas × 35 cols df_enc=395 filas × 51 cols></pre>

Por último, señalar en cuanto al registro del avance, se definen a grandes rasgos los pasos (Salinas, 2026a; Salinas, 2026b; Salinas, 2026c)

:

- Se reutilizó el pipeline funcional de F2.
- Se creó la clase base `PreprocesadorAsignatura`.
- Se implementaron clases hijas por asignatura.
- Se incorporó polimorfismo en `interpretar_resultados()`.
- Se agregó control de errores en `ejecutar_pipeline()`.
- Se validaron y exportaron los datasets procesados.

III. Repositorio GitHub (F3)

Organización del repositorio



Imagen 5. Estructura del repositorio

Commits descriptivos

El trabajo se realizó bajo la guía de la convención de commit, a modo de facilitar la identificación del tipo de cambio realizado en cada etapa, de manera tal de favorecer la trazabilidad, mantenibilidad y trabajo colaborativo del repositorio GitHub.

Actualiza README con sus ajustes correspondientes ConstanzaM0 committed 6 minutes ago	e85edd9
feat: F3_Eficiencia, validación y exportacion final Yenne Sepulveda committed 25 minutes ago	478b0bd
feat: F3_herencia_polimorfismo_rekursividad ConstanzaM0 committed 42 minutes ago	8a0edf4
feat: F3_Inicializar constructor, ejecución del método pipeline Juan de Dios Diaz Rios committed 1 hour ago	70bec14
feat: F3_creacion_notebook_clase_base_hijas_funciones ffarina11 committed 1 hour ago	98121d3
Actualizacion README con respecto a fase 3 ConstanzaM0 committed 2 hours ago	9e85229

Imagen 6. Captura desde GitHub del proyecto

El historial de commits permite verificar la evolución del núcleo algorítmico y la incorporación progresiva de los conceptos de Programación Orientada a Objetos, manteniendo la trazabilidad y reproducibilidad del proyecto.

Como estructura de secuencia de la F3 y los Commit generados, especificamos a continuación a través de la tabla de asociación.

Secuencia	Commit
Inicio de la Fase 3	feat: F3_creacion_notebook_clase_base_hijas_funciones
Migración del pipeline de F2	feat: F3_creacion_notebook_clase_base_hijas_funciones
Clase base	feat: F3_Inicializar constructor, ejecución del método pipeline
Clases hijas	feat: F3_Inicializar constructor, ejecución del método pipeline
Constructor	feat: F3_Inicializar constructor, ejecución del método pipeline
Encapsulamiento	feat: F3_Inicializar constructor, ejecución del método pipeline
Métodos del pipeline	feat: F3_Inicializar constructor, ejecución del método pipeline
Gráficos F3	feat: F3_Inicializar constructor, ejecución del método pipeline
Algoritmos	feat: F3_Eficiencia, validación y exportacion final
Exportación de resultados	feat: F3_Eficiencia, validación y exportacion final
Documentación	Actualiza README con sus ajustes correspondientes docs: F3_Documentación fases

Descripción tipos de prefijo:

feat	: Incorporación de nuevas funcionalidades o características.
refactor	: Reestructuración o mejora del código sin alterar su funcionalidad.
perf	: Mejoras de rendimiento y optimización.
fix	: Corrección de errores o fallos detectados.
docs	: Actualización de documentación, comentarios o archivos README.
build	: Cambios relacionados con dependencias o configuración del entorno.

README del repositorio

El repositorio GitHub del proyecto ([ffarina11/proyecto-grupo7-mcdi500](https://github.com/ffarina11/proyecto-grupo7-mcdi500)) dispone de un archivo README.md actualizado, el cual constituye la principal fuente de documentación técnica del proyecto (Universidad Andrés Bello, 2026). En él se describen el objetivo general, las instrucciones de instalación y ejecución, las dependencias registradas en requirements.txt y la organización de las carpetas y archivos. Esta documentación permite reproducir el entorno de trabajo y comprender la estructura del software desarrollado durante las fases F2 y F3.

El Readme presenta la descripción general del proyecto MCDI500 y su objetivo: Identificar y analizar los factores socioeconómicos y de preparación previa que explican las diferencias en el rendimiento académico entre estudiantes de dos establecimientos educacionales, mediante técnicas de análisis exploratorio y modelado predictivo sobre el Student Performance Dataset (Cortez y Silva, 2008).

En cuanto al contenido, éste se resume de la siguiente manera:

- Estructura completa del repositorio: carpetas data, notebooks, src, docs, .gitignore, README.md y requirements.txt.
- Instrucciones para reproducir el entorno: clonar repositorio, crear entorno virtual, activarlo, instalar dependencias y abrir Jupyter Lab.
- Descripción de la preparación inicial de datos en Fase 1.
- Explicación del pipeline de limpieza, transformación y validación de Fase 2.
- Descripción de la Fase 3, enfocada en núcleo algorítmico, POO, recursividad y eficiencia computacional.
- Información del dataset, fuente oficial, autores y archivos incluidos.


proyecto-grupo7-mcdi500
Public
Watch 0

main
5 Branches
0 Tags

Add file
Code

ConstanzaM0 Actualizacion README con respecto a fase 3		9e85229 · 36 minutes ago	33 Commits
data	F2_Ajuste_notebooks	4 days ago	
docs	Inicializa estructura del proyecto	2 weeks ago	
notebooks	F2_actualizar_notebook_limpieza	4 days ago	
src	F2_Resumen_Pipeline_Referencias_Anexos	5 days ago	
.gitignore	F2_cargainicial_exploracion	5 days ago	
README.md	Actualizacion README con respecto a fase 3	36 minutes ago	
requirements.txt	Actualiza librerias	last week	

README

Proyecto Grupo 7 — MCDI500

Factores socioeconómicos y de preparación previa asociados al rendimiento académico

 PYTHON
  3.10+
  JUPYTER
  LAB
  CURSO
  MCDI500

Descripción

Este repositorio contiene el proyecto transversal del curso MCDI500 — Programación para la ciencia de datos del Magíster en Ciencias de Datos e Inteligencia Artificial de la *Universidad Andrés Bello (UNAB)*.

El objetivo principal de esta investigación es analizar el *Student Performance Dataset* ([Cortez & Silva, 2008](#)) para identificar y evaluar qué factores socioeconómicos y de preparación previa explican de mejor manera las diferencias en el rendimiento académico entre estudiantes de dos establecimientos educacionales portugueses.

Imagen 7: Captura (fragmento) del README del proyecto.

IV. Notebooks ejecutables (F3)

1. Ejecución reproducible

El notebook F3_S02_Nucleo_Algoritmico_POO.ipynb puede ejecutarse completamente desde el inicio mediante la opción *Restart & Run All*, garantizando la reproducibilidad de los resultados.

```
resumen_f3.append(" HERENCIA Y POLIMORFISMO")
resumen_f3.append("     PreprocesadorMatematicas.interpretar_resultados() -> análisis específico MAT")
resumen_f3.append("     PreprocesadorPortugues.interpretar_resultados() -> análisis específico POR")
resumen_f3.append("")
resumen_f3.append(" RECURSIVIDAD")
resumen_f3.append("     merge_sort() ->     ordenar las listas dividiendolas en 2 ")
resumen_f3.append("     combinar() ->     una ambas listas ordenadas")
resumen_f3.append("")
resumen_f3.append(" EFICIENCIA")
resumen_f3.append("     determinar_aprobados: vectorizado ~[t_bucle/t_vec:.0f]x más rápido que iterrows")
resumen_f3.append("")
resumen_f3.append(" DATOS EXPORTADOS -> data/processed/F3/")
resumen_f3.append("     student_mat_clean.csv      (prep_mat.df.shape[0]) x (prep_mat.df.shape[1])")
resumen_f3.append("     student_mat_clean_encode.csv (prep_mat.df_enc.shape[0]) x (prep_mat.df_enc.shape[1])")
resumen_f3.append("     student_por_clean.csv      (prep_por.df.shape[0]) x (prep_por.df.shape[1])")
resumen_f3.append("     student_por_clean_encode.csv (prep_por.df_enc.shape[0]) x (prep_por.df_enc.shape[1])")
resumen_f3.append("     student_union_clean_encode.csv (df_union.shape[0]) x (df_union.shape[1])")
resumen_f3.append("     " * 65)

print("\n".join(resumen_f3))

=====
RESUMEN - NÚCLEO ALGORÍTMICO POO (FASE 3)
=====

OBJETOS CREADOS
prep_mat = <clases.PreprocesadorMatematicas object at 0x0000169038BE3F0>
prep_por = <clases.PreprocesadorPortugues object at 0x0000169038BE2C0>

PIPELINE ENCAPSULADO (heredado de PreprocesadorAsignatura)
cargar() -> explorar() -> limpiar() -> transformar() -> validar() -> exportar()

HERENCIA Y POLIMORFISMO
PreprocesadorMatematicas.interpretar_resultados() -> análisis específico MAT
PreprocesadorPortugues.interpretar_resultados() -> análisis específico POR

RECURSIVIDAD
merge_sort() ->     ordenar las listas dividiendolas en 2
combinar() ->     una ambas listas ordenadas

EFICIENCIA
determinar_aprobados: vectorizado ~211x más rápido que iterrows

DATOS EXPORTADOS -> data/processed/F3/
student_mat_clean.csv      395 x 36
student_mat_clean_encode.csv 395 x 54
student_por_clean.csv      649 x 36
student_por_clean_encode.csv 649 x 54
student_union_clean_encode.csv 1044 x 54
=====

8. Bibliografía (APA 7)

1. Cortez, P. (2008). Student Performance [Dataset]. UCI Machine Learning Repository. https://doi.org/10.24432/CSTG7T
2. Cortez, P., & Silva, A. (2008). Using data mining to predict secondary school student performance. University of Minho. https://doi.org/10.24432/CSTG7T
3. Dua, D., & Graff, C. (2019). UCI Machine Learning Repository. University of California, Irvine, School of Information and Computer Sciences. https://archive.ics.uci.edu
4. McKinney, W. (2022). Python for data analysis (3rd ed.). O'Reilly Media.
```

2. Integración con F2. Las funciones previamente validadas son invocadas desde los métodos implementados en las clases de la Fase 3.

```
# =====
# ENTORNO E IMPORTACIÓN DE MÓDULOS DEL PROYECTO
# =====
import sys, os
sys.path.append(os.path.abspath("../src"))

import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
import timeit

warnings.filterwarnings('ignore')
plt.rcParams['figure.figsize'] = (10, 4)
plt.rcParams['figure.dpi'] = 100
sns.set_theme(style='whitegrid', palette='muted')

# Funciones de La Fase 2 (pipeline funcional ya validado)
from functions import (
    cargar_dataset, mostrar_primeras_filas, resumen_dataset,
    validar_rangos, detectar_outliers_iqr, analizar_correlaciones,
    analizar_g3_cero, limpiar_dataset, winsorizacion,
    codificar_binarias, codificar_ohc, crear_variables_derivadas,
    validar_dataset_final, exportar_dataset, integrar_datasets, merge_sort,
    combinar_aprobado_bucle, aprobado_vectorizado
)

# Núcleo algorítmico de La Fase 3: clases (POO) y funciones recursivas
from clases import PreprocesadorAsignatura, PreprocesadorMatematicas, PreprocesadorPortugues

print("✓ Entorno listo. Módulos de Fase 2 (functions) y Fase 3 (clases) importados.")
```

3. Nuevos módulos POO

```
▼ src
  .gitkeep
  clases.py
  functions.py
  librerias.py
```

Imagen 8. Directorio de nuevos módulos POO

4. Clase base y herencia. Implementación de herencia mediante clases especializadas por asigna

```
Code Blame 320 líneas (269 loc) · 13.9 KB
52 # =====
53 class PreprocesadorAsignatura:
54     """Clase base que encapsula el pipeline completo de la Fase 2
55     (carga -> exploración -> limpieza -> transformación -> validación
56     -> exportación) para UNA asignatura del Student Performance Dataset.
57
58     Atributos de instancia
59     -----
60     ruta : str
61         Ruta al CSV crudo (data/raw/...).
62     asignatura : str
63         Identificador corto de la asignatura ('mat' o 'por').
64     df_raw : pd.DataFrame | None
65         Copia inmutable de los datos originales (tal cual se cargan).
66     df : pd.DataFrame | None
67         Copia de trabajo: se actualiza en limpiar().
68     df_enc : pd.DataFrame | None
69         Versión codificada/transformada (resultado de transformar()).
70
71     Atributo de clase
72     -----
73     NOMBRE_ASIGNATURA : str
74         Nombre legible de la asignatura. Se sobrescribe en cada
75         clase hija (PreprocesadorMatematicas, PreprocesadorPortugues).
76     """
77
78     NOMBRE_ASIGNATURA = "Genérica"
79
```

Imagen 9. Clase PreprocesadorAsignatura

```

263 # =====
264 class PreprocesadorMatematicas(PreprocesadorAsignatura):
265     """Especialización de PreprocesadorAsignatura para Matemáticas (student-mat.csv)."""
266
267     NOMBRE_ASIGNATURA = "Matemáticas"
268
269     def __init__(self, ruta: str = "../data/raw/student-mat.csv"):
270         # super().__init__ reutiliza el constructor de la clase padre
271         super().__init__(ruta=ruta, asignatura='mat')
272
273
274     def interpretar_resultados(self) -> None:
275         """Interpretación de las correlaciones con G3 específica de Matemáticas
276         (resultados obtenidos en la Fase 2 del proyecto)."""
277         print(f"\n{'='*60}")
278         print(f" INTERPRETACIÓN - {self.NOMBRE_ASIGNATURA}")
279         print(f"{'='*60}")
280         print("""
281         Las variables con mayor relación positiva con G3 corresponden a las
282         notas de períodos anteriores: G2 (0.90) y G1 (0.80), lo que indica
283         que el desempeño previo es el principal predictor del rendimiento final.
284
285         En menor medida, el nivel educativo de la madre (Medu=0.22) y del
286         padre (Fedu=0.15) muestran correlaciones positivas débiles.
287
288         La variable con mayor asociación negativa es 'failures' (-0.36):
289         los estudiantes con más asignaturas reprobadas previamente tienden
290         a obtener notas finales más bajas. Las variables de hábitos (goout,
291         Dalc, Walc) presentan asociaciones débiles.
292         """)
293

```

Imagen 10. Clase hija *PreprocesadorMatematicas*

5. Método `ejecutar_pipeline()`. Método de alto nivel encargado de orquestar las etapas de carga, exploración, limpieza, transformación, validación y exportación.

```
proyecto-grupo7-mcdi500 / src / clases.py
Code Blame 320 lines (269 loc) · 13.9 KB

205 # MÉTODO DE ALTO NIVEL - orquesta todo el pipeline
206 # -----
207 def ejecutar_pipeline(self, exportar_resultados: bool = True,
208                       base_path: str = "../data/processed/F3/") -> bool:
209     print(f"\n{'='*60}")
210     print(f" PIPELINE - {self.NOMBRE_ASIGNATURA} (objeto: {self.__class__.__name__})")
211     print(f"\n{'='*60}")
212
213     pasos = [
214         ("cargar", self.cargar),
215         ("explorar", self.explorar),
216         ("limpiar", self.limpiar),
217         ("transformar", self.transformar),
218         ("validar", self.validar),
219     ]
220
221     ok = False
222     for nombre_paso, metodo in pasos:
223         try:
224             resultado = metodo()
225             if nombre_paso == "validar":
226                 ok = resultado
227         except Exception as e:
228             print(f"\n❌ Pipeline detenido en el paso '{nombre_paso}': {type(e).__name__}: {e}")
229             print(f" Estado del objeto: {repr(self)}")
230             return False
231
232     if exportar_resultados:
233         try:
234             self.exportar(base_path)
235         except Exception as e:
236             print(f"\n⚠️ Validación exitosa pero falló la exportación: {type(e).__name__}: {e}")
237
238     return ok
```

Imagen 11. Método ejecutar_pipeline

6. Recursividad. Implementación del algoritmo recursivo Merge Sort con complejidad $O(n \log n)$.

```
MINGW64-C:/Users/yenne.se x F3_Nucleo_Algoritmico_POO x functions.py

532 # función de ordenamiento merge sort F3
533 def merge_sort(lista):
534     """
535     Ordena una lista usando Merge Sort recursivo (divide y vencerás,  $O(n \log n)$ ).
536     CASO BASE : lista de 0 o 1 elemento + ya ordenada, se retorna tal cual.
537     CASO RECURSIVO: divide la lista a la mitad, ordena cada parte
538                   recursivamente y combina los resultados.
539     """
540     if len(lista) <= 1: # CASO BASE: lista mínima, nada que ordenar
541         return lista
542
543     medio = len(lista) // 2
544     izq = merge_sort(lista[:medio]) # divide + ordena mitad izquierda
545     der = merge_sort(lista[medio:]) # divide + ordena mitad derecha
546     return combinar(izq, der) # y vence (combina)
547
```

Imagen 12. Implementación función merge_sort()

7. Comparación de complejidad

```

559
560 # Función para determinar aprobado usando un bucle fila a fila F3*
561 def aprobado_bucle(df: pd.DataFrame) -> list:
562     """
563     Determina si cada estudiante aprobó (G3 >= 10) usando un bucle fila a fila.
564     Complejidad: O(n) – pero lento por el overhead de Python en cada iteración.
565     """
566     resultado = []
567     for _, fila in df.iterrows():      # iterrows es lento: crea un objeto por fila
568         resultado.append(1 if fila["G3"] >= 10 else 0)
569     return resultado
570
571 # Función para determinar aprobado usando operación vectorizada F3*
572 def aprobado_vectorizado(df: pd.DataFrame) -> pd.Series:
573     """
574     Determina si cada estudiante aprobó (G3 >= 10) con operación vectorizada.
575     Complejidad: O(n) – pero rápido porque opera en C/NumPy de una sola vez.
576     """
577     return (df["G3"] >= 10).astype(int)
578
579

```

Imagen 13. Funciones bucle y vectorización.

8. Resultados

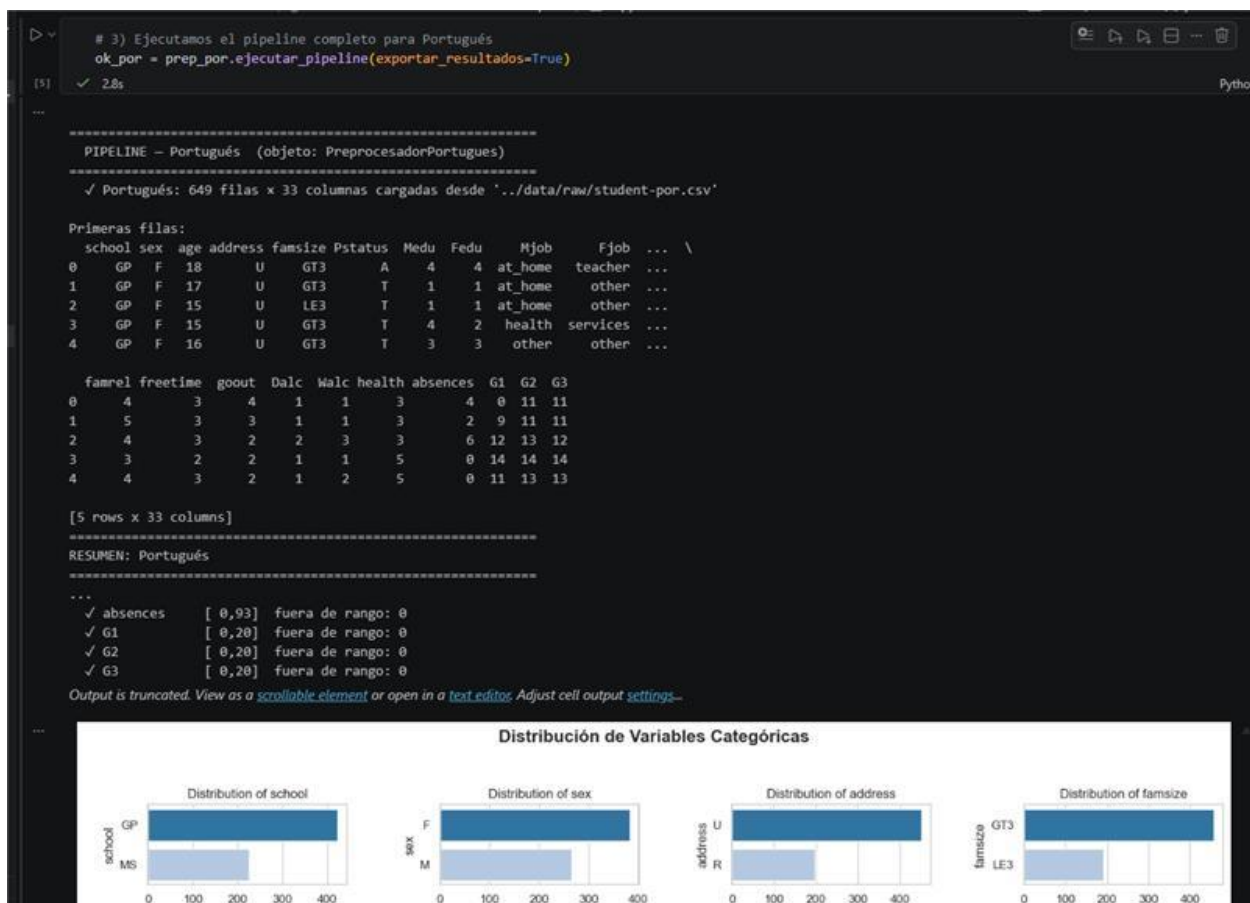


Imagen 14. Dataset exportado, muestra de gráficos generados.

```
# Polimorfismo: mismo metodo, comportamiento distinto segun la clase concreta
prep_mat.interpretar_resultados()
prep_por.interpretar_resultados()

✓ 0.0s Python
```

```
=====
INTERPRETACIÓN – Matemáticas
=====

Las variables con mayor relación positiva con G3 corresponden a las
notas de períodos anteriores: G2 (0.90) y G1 (0.80), lo que indica
que el desempeño previo es el principal predictor del rendimiento final.

En menor medida, el nivel educativo de la madre (Medu=0.22) y del
padre (Fedu=0.15) muestran correlaciones positivas débiles.

La variable con mayor asociación negativa es 'failures' (-0.36):
los estudiantes con más asignaturas reprobadas previamente tienden
a obtener notas finales más bajas. Las variables de hábitos (goout,
Dalc, Walc) presentan asociaciones débiles.

=====
INTERPRETACIÓN – Portugués
=====

Igual que en Matemáticas, las notas previas dominan la correlación
con G3: G2 (0.92) y G1 (0.83) son los predictores más fuertes.

A diferencia de Matemáticas, aquí el tiempo de estudio (studytime=0.25)
y el nivel educativo de la madre (Medu=0.24) tienen un peso algo mayor.

El consumo de alcohol entre semana (Dalc=-0.20) y fin de semana
(Walc=-0.18) se asocia negativamente con el rendimiento, más
```

Imagen 15. Interpretaciones finales análisis dataset matemáticas y portugués

V. Bibliografía (APA 7)

Cooper, J. W. (2021). *Python programming with design patterns*. Addison-Wesley Professional.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to algorithms* (4th ed.). MIT Press.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature*, 585(7825), 357–362. <https://doi.org/10.1038/s41586-020-2649-2>

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3), 90–95. <https://doi.org/10.1109/MCSE.2007.55>

NumPy Developers. (2024). *NumPy documentation*. <https://numpy.org/doc/stable/>

Python Software Foundation. (2026). *The Python Standard Library*. <https://docs.python.org/3/library/>

Richards, M., & Ford, N. (2025). *Fundamentals of software architecture: A modern engineering approach* (2nd ed.). O'Reilly Media.

Salinas, O. (2026a). *Avance Fase 3 — Semana 2: Núcleo algorítmico y Programación Orientada a Objetos* [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Salinas, O. (2026b). *Programación orientada a objetos (POO) aplicada a la ciencia de datos* [Infografía]. Universidad Andrés Bello, Santiago, Chile.

Salinas, O. (2026c). *Programación para Ciencia de Datos: Parte Práctica de la Semana 2 — Clases, objetos, herencia y namespaces* [Apunte]. Universidad Andrés Bello, Santiago, Chile.

Salinas, O. (2026d). *Tipos, variables, booleanos, condicionales, ciclos, funciones, módulos y excepciones en Python* [Apunte]. Universidad Andrés Bello, Santiago, Chile.

The pandas development team. (2024). *pandas documentation*. <https://pandas.pydata.org/docs/>

Universidad Andrés Bello. (2026). *Guía de Git y GitHub: Entorno de desarrollo con Python y JupyterLab. Versión para Windows 10 y 11* [Material de apoyo].